

İTÜ Rover Araştırma Ödevi

Ali Zahid Erenoğlu

1 ROS

ROS nedir

Gerçek işletim sisteminin üzerinde çalışan ara katman yazılımıdır (middleware). Robot üzerindeki farklı yazılım parçalarının birbiriyle standart bir dilde konuşmasını sağlayan iletişim ve yönetim altyapısıdır.

Nasıl çalışır

Robot yazılımını küçük ve bağımsız modüler programlara (node) böler. Bu programlar birbirlerine mesajlar göndererek iletişim kurar.

ROS 1 ile ROS 2 farkı: ROS 1’de iletişim, merkezi bir yönetici tarafından ROS Master üzerinden kontrol edilir. ROS 2’de DDS (Data Distribution Service) adı verilen daha modern, merkeziyetsiz ve güvenilir bir altyapı kullanarak uçtan uca doğrudan iletişim sağlar.

Neden node’ları kullanıyoruz

Tek bir devasa programda toplamama sebeplerimiz:

- Çökme riski: Görüntü işleme bloğunda oluşan basit hata, tüm programın çökmesine sebep olabilir.
- Geliştirme ve hata bulma zorluğu: Yazılım ekibindeki 5 kişinin aynı anda tek bir dosyada çalışması durumu zorlaştırır. Küçük bir değişiklikte bütün robotu denemek zaman alır.
- Yeniden kullanılabilirlik: Kodu başka bir araca taşımak imkânsızlaşır eğer tek bir program hâline getirirsek.

ROS, modüler yapısı sayesinde modül çökse bile diğerlerinin çalışmaya devam etmesini sağlar. Otonom sistemler için alternatif haberleşme çatıları olsa da araç kütüphanesi ve donanım desteği sebebiyle rover için vazgeçilmez bir yapı olur.

Rover’ı nerede kullanabiliriz

- LiDAR programı: Sadece lazer verisini okur ve ağa yayılır.
- Kamera programı: Görüntüyü alır ve ağır görüntü işleme algoritmaları koşturur.

Middleware nedir

Normalde iki farklı programın veya iki farklı bilgisayarın haberleşmesi için düşük seviyeli ağ yapıları kullanılır ama middleware bu karmaşık yapıların üzerine abstraction layer ekler.

Veri dönüşümü serialization ve marshalling

Bir program C++ ile yazılmış bir struct tutuyor, diğeri Python’da bir dictionary tutuyor olsun. Middleware bu veriyi ortak bir formata çevirip karşı tarafa iletir, orada tekrar hedef dile çevirir.

Protokol ve adres yönetimi

Bir programın diğerinin IP adresini, port numarasını ya da bellekteki tam adresini bilmesine gerek kalmaz. Middleware bu eşleştirmeleri arkada dinamik olarak çözer.

Neden gereklidir

Middleware olmadan bir otonom sistemi yazmaya kalkarsak şu engellerle karşılaşırız:

Farklı diller ve donanımlar: Robot üzerinde görüntü işleme için Python kullanırken gerçek zamanlı motor kontrolü veya SLAM için C++ kullanmak isteriz. Middleware olmadan bu iki programın kendi aralarında veri paylaşması zordur.

Tekerleği yeniden icat etmek istemeyiz. Her projede sıfırdan socket bağlantısı, paket kaybı kontrolü gibi şeyleri kodlamak yerine test edilmiş ve verimli hâle getirilmiş iletişim katmanı kullanabiliriz.

Loose coupling: Kod parçaları birbirine kesin bağımlı olmaz. Veriyi gönderen LiDAR programı, o veriyi kimin dinlediğini bilmek zorunda kalmaz, middleware'e bırakır.

Neden alternatifleri kullanmak yerine ROS kullanıyoruz

ROS sadece haberleşme kütüphanesi değildir.

ZeroMQ, MQTT sadece veriyi A noktasından B noktasına taşır fakat robot için sadece bu görev yeterli değildir.

- Veriyi taşıdıktan sonra görselleştirmek gerekir: RViz.
- Robotu bilgisayarda simüle etmek gerekir: Hazır Gazebo var.
- Sensör verilerini kaydedip tekrar oynatmak gerekir: rosbag var.
- Farklı sensörlerin birbirine göre uzaydaki konumlarını ve dönüşümlerini hesaplamak gerekir: Hazır tf2 kütüphanesi var.

Sonuç olarak ZeroMQ vb. kullanırsan haberleşmeyi çözersin ama diğer imkânları elde etmek için uzun bir süre kodlama ve emek gerekir.

Hazır sensör ve donanım sürücüler

Piyasadaki neredeyse bütün endüstriyel LiDAR, derinlik kamerası veya IMU üreticisi doğrudan resmî ROS sürücüsü (ROS driver) yayımlar. ROS kullanıldığında LiDAR'ı takip tek bir satırla veriyi okuyabilirsin.

Standartlaşmış mesaj tipleri

ROS'ta sensör verileri evrenseldir.

Bütün 2B LiDAR'lar sensor_msgs/LaserScan tipinde veri üretir.

ROS'ta açık kaynak olarak hazır algoritmalar bulunur

Bir cerrahi kol gibi, 1 milisaniyelik gecikme bile kabul edilmezse Orocos tercih edilir.

Çok düşük donanım kaynağı, mesela çok küçük bir mikrodenetleyici varsa, ROS kuracak Linux alanı yoksa doğrudan ZeroMQ, LCM kullanılabilir.

Package ve node

Package

Bilgisayarın sabit diskinde duran, belirli bir robotik göreve ait tüm dosyaların bir arada tutulduğu organize klasördür.

Node

ROS sistemine katılan, belirli tek bir mantıksal görevi yerine getiren ve ROS haberleşme kanallarını kullanabilen yazılım birimidir.

Topic publisher ve subscriber

Topic

ROS düğümlerinin birbirine veri göndermek için kullandığı tek yönlü ve isimlendirilmiş bir veri iletim ağıdır.

Publisher

Belirli bir topic üzerine sürekli veya aralıklarla veri mesajı basan arayüzdür.

Subscriber

İlgilendiği topic'e gelen verileri anlık olarak dinleyen yazılım arayüzüdür.

Nasıl çalışır

İletişim, publish/subscribe mimarisiyle kurulur.

Bir düğüm, ROS ağına “/scan isimli topic’e veri göndereceğim.” diyerek publisher tanımlar.

Başka bir düğüm, “Ben /scan isimli topic’e gelecek verileri dinlemek istiyorum.” diyerek bir subscriber açar.

ROS altyapısı da bu iki ucu arka planda birbiriyle eşler (TCP veya UDP üzerinden).

Publisher veriyi hatta basar. O hatta o anda abone olan node’ların hepsi bu veriyi eş zamanlı kendi hafızasına alır.

En kritik özelliği loose coupling: Publisher, verisini kimin okuduğunu, hatta hiç okundu mu, okunmadı mı bilmez. Subscriber, verinin hangi dilde yazılmış vb. bilgilerini önemsemez, sadece hatta gelen veriyi alır.

Callback mekanizması

Bir subscriber node’a abone olduğu topic’ten yeni bir mesaj ulaştığında ROS tarafından otomatik tetiklenen fonksiyondur.

Normal şartlarda kod yukarıdan aşağı doğru çalışır ancak sensörün ne zaman veri üreteceğini bilemezsin.

Kodunun içinde abone tanımlarken ona bir fonksiyon bağlarsın:

```
rospy.Subscriber("/scan", LaserScan, lidar_callback)
```

Program arkada rospy.spin() satırına gelir ve beklemeye geçer.

LiDAR donanımı her yeni taramayı tamamlayıp hatta yeni bir LaserScan mesajı bastığı anda ROS arka planda bu paketi teslim alır, içindeki lazer verisini çıkarıp yazdığımız lidar_callback fonksiyonunun içine koyar ve o fonksiyonu çalıştırır.

Fonksiyon içindeki kod çalışır, hesaplama yapılır ve fonksiyon sonlanır. Sonraki mesaja kadar kod yeniden uykudadır.

Neden gerekiyor

Callback olmasaydı kodun içine sonsuz while True döngüsü koyup “Veri geldi mi?” diye kontrol etmemiz gerekirdi. Bu da polling (meşgul bekleme) yaratır ve bilgisayarın işlemcisi %100 yük altına sokar, aşırı ısınır.

Başka hesaplamalar gecikirdi.

```
def lidar_callback(msg):
```

Yeni lazer taraması geldiğinde bu fonksiyon çalışır, engelin yönü güncellenir ve motor komutu üretilir.

Service

ROS service, iki node arasında request/response mantığıyla çalışan iki yönlü bir haberleşme modelidir.

Server (hizmeti sunan node): Ağda bekler. “İstek gelirse şu işlemi yapacağım ve sonucu döneceğim.”

Client (hizmet talep eden node): İhtiyaç duyduğu anda sunucuya bir istek gönderir ve sunucunun döndürdüğü yanıtı alır.

Service ve topic

Topic’te iletişim yönü tek yönlüdür, service’te 2 yönlüdür.

Sıklık: Topic, sürekli kesintisiz veri akışı. Service, ihtiyaç duyulduğunda tetikleme ile.

Geri bildirim: Topic, yok. Service, “Başarılı, başarısız, bitti mi?” yanıtı döner.

Launch dosyası

Tek bir komutla birden fazla düğümü, bu düğümlerin ihtiyaç duyduğu ayarları ve simülasyon ortamını topluca başlatan yapılandırma dosyasıdır.

Bir rover’ı çalıştırmak için 5–6 farklı programı çalıştırmak gerekir. Launch dosyasıyla düğümleri ve ayarlarını tek komutla başlatır.

XML veya Python formatında yazılmış tek bir metin dosyasıdır.

Parametre

Bir programın koduna dokunmadan çalışma anında davranışını dışarıdan değiştirebilmeni sağlayan ayar düğmesidir.

rosvbag

Ağda o anda akmakta olan topic mesajlarını zaman damgalarıyla birlikte diske .bag kaydeden ve daha sonra bu dosyayı açıp mesajları birebir aynı sıklıkta tekrar ağa basabilen ROS aracıdır. Robotun kara kutusu veya kayıt cihazı denilebilir.

Neden gerekir

Sahada robot garip bir hata yapıp engele çarptığında, o anki tüm veriler bag dosyasına kaydedilmişse kaydı tekrar oynatarak yazılan kodun 1 saniyede gelen veriye nasıl tepki verdiği adım adım incelenebilir ve hata çözülür.

rospy.loginfo

Günlükleme mantığıyla kullanılır.

print() fonksiyonu gibi, zaman damgasıyla terminale ve ROS'un merkezi loglama kanallarına yazdıran fonksiyondur.

Gazebo nedir

İçinde gerçekçi fizik motoru barındıran üç boyutlu dünya simülatörüdür.

Sanal bir evren kurar, fizik kuralları koyar: Yer çekimi, sürtünme, kütleler, eylemsizlik momentleri, nesnelerin birbirine çarpma dinamikleri.

Fiziksel dünyayı ve robotun donanımını simüle eder.

RViz

RViz (ROS Visualization) simülatör değildir. RViz, ROS topic'lerinden geçen verileri ekranda 3 boyutlu grafiklere dönüştüren sensör ve algoritma izleme aracıdır.

Robotun zihnini, ne algıladığını gösterir.

Robot modeli URDF Xacro ve world dosyası

World dosyası

- Güneş açısı, şiddeti.
- Yer çekimi ivmesi.
- Zeminin engebesi, sürtünmesi ve dokusu, ortamdaki sabit veya hareketli engeller.

Robot modeli URDF Xacro ve SDF

Ortamdan bağımsız, sadece robotun kendi fiziksel ve mekanik yapısını tanımlar.

- Link: Uzunluklar.
- Joint: Eklemler.
- Sensör montaj yerleri.

2 Sensörler

LiDAR

Nedir ve neyi ölçer

LiDAR, lazer ışığı kullanarak çevredeki yüzeylere olan mesafeyi ölçen sensördür. Açılımı Light Detection and Ranging.

LiDAR çalışma prensibi, "Önümde duvar var." sonucu var demek değil, belirli yönlerde mesafe ölçümleri verir. Bu ölçümlerden anlam çıkaran yazılımdır ve engelin nerede olduğuna karar verir.

2B LiDAR

Bir düzlem boyunca farklı açılarla ölçüm yapar. Tarama düzleminin altında veya üstünde kalan engeli doğrudan algılayamaz.

- Sensör bir lazer darbesi gönderir.

- Işıık bir yüzeye arpar. Bir kısmı geri döner.
- Sensör, gönderme ile geri dönüş arasındaki süreyi ölçer.

$$\text{Mesafe} = \text{ışık hızı} \times \text{geen süre} / 2$$

Robotta engel algılama ve engelden kaçınma, haritalama, konum belirleme için kullanılır.

Hata kaynakları ve sınırları

Yüzeyin özelliđi, hava koşulları ve sensörün görüş alanı sonucu etkiler.

- Cam ve parlak yüzeyler.
- Işıđı az yansıtan yüzeyler.
- Yağmur, sis, toz.
- Görüş alanı ve tarama düzlemi.
- Menzil ve ölçüm belirsizliđi.

LiDAR'ın sınırları vardır. En yakın ve en uzak ölçülebilme sınırı vardır. Bu sınırlar dışındaki nesneler için ölçüm alınamayabilir.

IMU

Nedir ve neyi ölçer

Açılımı Inertial Measurement Unit, Türkesi ataletsel ölçüm birimi.

Robotun hareketini takip etmek için kullanılan sensör grubudur.

Temelde 2 sensör içerir: İvmeöler, jiroskop.

İvmeöler 3 eksen de ölçüm yapar. Ölümde yer çekiminin de etkisi hesaba katılmalıdır.

Jiroskop 3 eksen de açısal hızı ölçer. Robotun hangi yönde, hangi hızla döndüğünü ölçer.

Bu ikisini barındıran IMU'lar 6 eksenli olarak geçer.

Bazı IMU'larda manyetik alanı ölçen manyetometre de bulunur. Pusula yönünü tahmin etmeye yardımcı olur.

Manyetometre içerenler 9 eksenli olarak geçer.

IMU nasıl alışır

Küçük IMU'ların çoğunda MEMS teknolojisi bulunur.

MEMS, ip üzerinde üretilen çok küçük mekanik yapıların elektronik devrelerle birlikte kullanılmasıdır.

İvmeöler

İinde esnek bağlantılarla tutulan küçük bir kütle vardır. Sensör ivmelendiğinde kütle gövdeye göre yer deđiştirir. Kütle ile sabit plakalar arasındaki elektriksel kapasite deđiştir. Bu deđişim ölçülerek ivme bilgisi elde edilir.

Jiroskop

İindeki küçük bir kütle sürekli titreştirilir. Sensör döndüğünde Coriolis etkisi, kütlede titreşim yönüne dik bir hareket oluşturur. Bu hareketin oluşturduğu kapasite deđişiminden açısal hız hesaplanır.

Manyetometre

Manyetik alanı ölçer. Hall etkisini kullanan tiplerde, üzerinden akım geçen bir malzemede manyetik alan nedeniyle yükler yana sapar ve bir gerilim oluşur. Bu gerilimden manyetik alan bilgisi elde edilir. Bazı tiplerde manyetodirenç kullanılır ve bunlarda malzemenin manyetik alanla değişen elektriksel direnci hesaplanır.

Nerede kullanınız

- Eğim takibi.
- Dönüş kontrolü.
- Konum tahminine destek: IMU verileri, tekerlek enkoderi, GNSS veya LiDAR verileriyle birleşerek robotun hareketi ve konumu tahmin edilir.

Robotun yönelimi anlatılırken 3 açı kullanılır: Roll, pitch, yaw.

- Roll: Sağa sola yatma.
- Pitch: Ön tarafın yukarı veya aşağı eğilmesi.
- Yaw: Sağa veya sola dönerek baktığı yönü değiştirme.

Sensör verilerini birleştirme sebebimiz

Tekerlek patinaj yaptığında tekerlekteki dönüşü ölçen enkoderin beklediği kadar dönmemiş olabilir. IMU ve diğer sensörler, hareket tahminini kontrol etmek için ek bilgi sağlar.

Bu işleme sensör füzyonu denir.

Amaç, farklı sensörlerin güçlü yönlerinden yararlanıp ölçüm hatalarının etkisini azaltmaktır.

Bias ölçümdeki sistematik kayma

Robot dönmüyorken jiroskoptan alınan veri 0 göstermesi gerekirken 0.1 gösterebilir. Bu ölçüm kaymasına bias denir.

Sıcaklık ve çalışma süresi değiştikçe bias değişebilir.

Drift hesaplanan açı ve konumun zamanla sapması

Açısal hızı zaman boyunca biriktirerek açı değişimi hesaplıyoruz. Ölçümdeki hata da bu sırada birikir.

Konum hesabında sorun daha büyük olur. IMU'nun ölçtüğü ivme kullanılarak hız, ardından konum hesaplanır. İvme ölçümündeki küçük hatalar ölçümde birikir ve zamanla konumun yanlış hesaplanmasına neden olur. Bu yüzden konum tahmini, GNSS gibi başka sensörlerden alınan bilgilerle düzeltilir.

Gürültü titreşim ve sıcaklık

Gürültü: Gerçek hareket değişmese bile ölçümde rastgele küçük dalgalanmalar oluşur.

Titreşim: Motorlar ve engebeli zemin, izlemek istediğimiz genel harekete titreşimler IMU ölçümlerini etkileyerek hareket tahmininde hatalara neden olabilir.

İvme ile eğimin karışması

İvmeölçer verisinden eğim tahmin ederken robotun hareketini de dikkate almak gerekir.

Manyetik etkiler

Manyetometre bulunan birimlerde motorlar, mıknatıslar ve manyetik alanı bozan metal parçalar yön tahminini etkileyebilir.

GNSS

GNSS nedir ve GPS ile farkı nedir

GNSS, Global Navigation Satellite System ifadesinin kısaltmasıdır. Uydulardan gelen radyo sinyalleriyle konum, hız ve zaman bilgisi elde edilmesini sağlar.

GNSS genel addır. GPS ise bu kapsamdaki sistemlerden biridir:

- GPS: ABD'nin sistemi.
- Galileo: Avrupa Birliği sistemi.
- GLONASS: Rusya'nın sistemi.
- BeiDou: Çin'in sistemi.

Robot üzerindeki GNSS alıcısı, uydu sinyallerini işleyerek enlem, boylam ve yükseklik bilgilerini hesaplar. Rover'da konum belirleme ve hedef koordinata ilerleme amacıyla kullanılır.

GNSS konumu nasıl hesaplar

- Uydu, zaman bilgisi, yörünge bilgilerini içeren radyo sinyalleri yayınlar.
- Alıcı, bu bilgilerden uydunun sinyali gönderdiği andaki konumunu hesaplar.
- Sinyalin kendisine ulaşma süresinden uyduya olan mesafeyi tahmin eder.
- Birden fazla uydudan aldığı ölçümleri birlikte kullanarak kendi konumunu hesaplar.

Bir uyduya olan uzaklığı bilmek, tam olarak nerede olduğunu söylemez. Aynı uzaklıkta pek çok nokta bulunur. Diğer uydulardan gelen mesafe bilgileri olası konumları sınırlar.

Standart üç boyutlu konum hesabında dört bilinmeyen vardır:

- Alıcının x konumu.
- Alıcının y konumu.
- Alıcının z konumu.
- Alıcının saat hatası.

Alıcının saati, uydu saatleriyle tam eşleşmez. Bu fark da mesafe hesabını etkiler. Alıcı en az dört uygun uydu ölçümünü birlikte kullanarak konumunu ve saat hatasını çözer.

Zaman hatası neden bu kadar önemli

Işık hızı çok hızlı olduğu için 1 milisaniye bile yaklaşık 300 kilometre farklılık gösterir.

Rover için kullanımı

GNSS, açık alanda çalışan robotun dünya üzerindeki konumunu belirlemede kullanılır.

Rover için 3 görevi yerine getirir:

- Hedef koordinata ilerleme: Robot, mevcut konumundan istenilen koordinata ulaşmak için iki konumu kıyaslar, hedefe uzaklığını ve hangi yönde olduğunu hesaplar.
- Rota takibi: Önceden belirlenmiş koordinatları sırayla takip etmekte kullanılır.
- Hareket kaydı: Robotun hangi konumlarda bulunduğunu kaydeder ve rotası incelenir.

Hangi durumlarda GNSS hatalı ya da eksik konum verir

Sinyalin engellenmesi durumlarında: Tüneller, binalar, yoğun ağaç bölgeleri. Bu nedenle açık gökyüzü gerekir.

Sinyalin yansımaları: Uydu sinyali doğrudan geldiği yolun üzerindeki yüzeylerden yansıtılarak da antene ulaşabilir. Buna çok yollu yayılım (multipath) denir.

Uyduların gökyüzündeki konumları

Yalnızca uydu sayısı önemli değildir. Ayrıca uyduların farklı yönlerde dağılmış olmaları, konumu belirlemede ölçümün daha iyi olmasını sağlar.

Birbirine yakın olan uydularda benzer mesafe hataları daha büyük konum hatasına sebep verebilir.

Atmosfer etkisi

Uydu sinyalleri atmosferden geçerken gecikir. Bu gecikmenin hesaba katılması gerekmektedir. Aksi durumda mesafe hatası oluşur.

Rover'da etkisi

Rover'ın durduğu durumda koordinat hataları sebebiyle robotun hareket ettiğini düşünüp düzeltmeler yapılabilir.

Bu yüzden GNSS konumu değerlendirilirken ölçüm güvenilirliği hesaba katılır. IMU ve enkoder gibi sensörlerin hesaba katılması konum tahmininde güveni artırır.

Derinlik kamerası

Görüş alanındaki yüzeylerin derinliğini ölçen kamera sistemidir.

Robotun gördüğü nesnelerin ne kadar uzakta olduğunu anlamasına yardımcı olur.

Normal kamera ile farkları

Normal kameranın aksine renk ve parlaklık bilgisi vermez, görüntüdeki noktalara ait derinlik bilgisi verir.

RGB D nedir

Bazı kameralar renkli görüntüyü ve derinlik bilgisini birlikte verir.

- RGB: Kırmızı, yeşil ve mavi.
- D: Depth, derinlik.

Rover'da ihtiyacı karşılama: Robotun önündeki engelin sadece görüntüsü değil, konumunu ve 3 boyutlu şeklini de değerlendirebilir. Örneğin bir engelin yüksekliğini ve geçiş alanını hesaplamak için bu verilerden yararlanır.

Nasıl çalışır

3 yaygın yöntem vardır: Stereo görüş, yapılandırılmış ışık ve uçuş süresi (ToF).

Stereo görüşte yan yana iki kamera bulunur. Kameralar aynı yüzeye farklı konumlardan baktığı için yüzeydeki bir nokta iki görüntüde farklı yatay konumlarda görünür.

Yapılandırılmış ışık

Bu yöntemle kamera sistemi karşısındaki yüzeye bilinen bir ışık deseni yansıtır. Bu desen noktalar veya çizgilerden oluşabilir. Genellikle insan gözünün göremediği kızılötesi kamera kullanılır.

Projektör ışık desenini yansıtır, kamera yüzeyde deseni görüntüler.

Yüzeyin uzaklığı değişikçe kamera deseni farklı görüntüler.

Sistem bu kaymalardan derinliği hesaplar. Bu geometrik hesaplama yöntemine üçgenleme denir.

Güçlü güneş ışığı desenin algılanmasını zorlaştıracığından ölçümün güvenilirliği azalır.

Time of Flight

Işığın nesneye gidip geri dönmesini ölçerek uzaklığı belirler. Işık kameraya dönmeden önce birden fazla yüzeyden yansır daha uzun yol alır. Sistem bunu yanlış algılayabilir ve yapılan ölçümde hata oluşmasına sebep olabilir.

Derinlik kamerası, engeli algılama ve engelden kaçmak için, çevresini 3 boyutlu haritalamada, robot koluyla nesne tutmada, nesnenin 3 boyutlu noktasını bulmada kullanılabilir.

Encoder

Encoder nedir ve neden kullanılır

Döner encoder, bir milin açısal konumunu veya ne kadar döndüğünü ölçer.

Doğrusal encoder, bir parçanın düz bir hat üzerindeki konumunu veya yer değiştirmesini ölçer.

Motora verilen aynı komutlara rağmen milin dönüş hızı her zaman aynı değildir.

Bu sebeple encoder, gerçekleşen dönüşü ölçerek kontrol sistemine geri bildirim sağlar.

Kontrolcü, hedef hız ve ölçülen hızı karşılaştırarak motora komut gönderir.

- Dönüş miktarından açısal yer değişimi hesaplanır.
- Dönme hızı hesaplanır.
- Uygun encoder kullanımında dönüş yönü belirlenir.
- Tekerlek dönüşünden robotun ilerleme miktarı tahmin edilir.

Optik ve manyetik algılama

Optik encoder'da mile bağlı bir disk vardır.

Disk üzerinde ışığı geçiren ve engelleyen bölümlerle, bir taraftaki ışık kaynağından diğer taraftaki algılayıcının ışık algılamasına göre milin ne kadar döndüğünü hesaplar.

Manyetik encoder'da ise mile bağlı mıknatıs veya manyetik halka bulunur. Mil döndükçe sensörün algıladığı manyetik alan değişir. Sensör bu değişimi dönüş bilgisine dönüştürür.

Artımlı encoder ve mutlak encoder farkı

Artımlı encoder: Mil döndükçe darbeler üretir. Kontrolcü de bu darbeleri sayarak başlangıç noktasına göre ne kadar döndüğünü hesaplar.

Mutlak encoder: Ölçüm aralığındaki her ayırt edilebilir noktaya farklı bir kod verir. Bu yolla başlangıçtan itibaren darbe saymadan mevcut konumu okuyabilir.

Örneğin robot eklemi 70 derecedeyken kapatılıp açıldığında kalibre edilmiş mutlak encoder açılışta bu konumu bildirebilir.

Motor hız kontrolünde, robot kolunda eklemlerin dönüş açısını belirlemede, tekerlek odometrisi: Tekerlek dönüşlerinden robotun konum ve yön değişimi tahmin edilir.

Hata kaynakları

- Kayma ve patinaj.
- Tekerlek çapının yanlış bilinmesi: Lastik aşınması, basınç ve yük nedeniyle tekerleğin etkin yuvarlanma yarıçapı da değişir.
- Dişli boşluğu: Özellikle yön değiştirirken motor mili bir miktar döndüğü hâlde çıkış mili hemen hareket etmeyebilir.
- Sınırlı çözünürlük: Encoder çok küçük hareketleri ayrı ayrı algılayamayabilir.
- Sinyal sorunları: Elektriksel gürültü fazla sayım oluşturabilir.
- Montaj ve çevre koşulları.

3 Lokalizasyon

Lokalizasyon nedir

Lokalizasyon, robotun belirli bir koordinat sistemine göre nerede olduğu ve hangi yöne baktığını tahmin etmesidir.

Düz bir zeminde hareket eden rover için temel bilgiler:

- x: Bir eksendeki konumu.
- y: Diğer eksendeki konumu.
- Robotun baktığı yönü gösteren açı.

Konum ve yönelimin birlikte ifade edilmesine pose denir.

Her sensör konum tahminine farklı bilgi sağlar:

- Encoder, tekerlek dönüşlerinden robotun ne kadar ilerlediğini tahmin etmeye yardımcı olur.
- IMU, dönüş ve ivme bilgisi sağlar.
- GNSS, uygun koşullarda dünya üzerindeki konumu.
- LiDAR veya kamera, çevredeki özelliklerin bilinen harita ile eşleşmesini sağlar.

Robot, önceki konumunu ve son hareketine ait ölçümleri kullanarak yeni konumunu tahmin eder.

Daha sonrasında hareketten elde ettiği tahmini başka ölçümlerle karşılaştırır.

GNSS varsa hesapladığı konumdan yararlanır, bilinen harita varsa LiDAR veya kamera ölçümlerini haritayla karşılaştırır.

Sensörler farklı sonuç verirse hangisine güvenilir

Ölçümlerin belirsizlikleri dikkate alınır. Belirsizliği düşük olan ölçüme genellikle daha fazla ağırlık verilir, hatalı olduğu belirlenenler dışlanabilir.

Bu işlem sensör füzyonudur. Verileri yalnızca toplamak yerine ölçümlerin zamanı, yönelimi ve hataların özellikleri değerlendirilir.

4 SLAM

Simultaneous Localization and Mapping.

Eş zamanlı lokalizasyon ve haritalama.

Robotun daha önceden haritasını bilmediği bir çevrenin haritasını yaparken aynı anda bu harita üzerindeki konumu ve yönelimini tahmin etmesine denir.

Robot, LiDAR veya kamera gibi farklı sensörlerle çevresini algılar. Hareket ettikçe yeni ölçümler ile eski ölçümleri bağdaştırarak kendi konum tahminini günceller.

Otonom sürüşte SLAM'in ürettiği harita ve konum bilgisi, engelleri belirlemede ve bunlardan kaçınma planı yapılmasında sisteme girdi sağlar.

5 Otonom sürüş

Path planning

Path planning (yol planlama), otonom aracın başlangıç merkezinden hedef noktaya engellere çarpmadan ulaşmasını sağlayacak yolu hesaplama işlemidir.

Planlanan yol için çeşitli kriterler konulur: Kısa mesafe, güvenlik, enerji tüketimi ve aracın hareket kısıtları.

2 seviyede yapılır:

Global planlama

Sahip olunan haritanın tamamı kullanılarak başlangıç noktasından hedef noktaya planlama yapılır ve rota oluşur. A*, Hybrid A*, RRT/RRT*.

Lokal planlama

Global rotayı takip ederken sensörlerden gelen anlık verilerle rota güncellenir.

Örnek: DWA (Dynamic Window Approach), TEB, MPC tabanlı planlayıcılar.

Costmap

Costmap (maliyet haritası).

Aracın çevresi hücrelere bölünür ve her hücreye o bölgeden geçmenin maliyeti atanır.

Boş alanlar düşük maliyetlidir, engellere yakın bölgeler yüksek maliyetlidir.

Engelin bulunduğu bölgeler ise lethal olarak işaretlenir.

Path planning algoritmaları da en düşük toplam maliyetli yolu harita üzerinden seçer.

ROS/Nav2'de costmap katmanlı yapıdadır:

- Static layer: Önceden bilinen harita.
- Obstacle layer: LiDAR, kamera gibi sensörlerden anlık verilerle bulunan engeller.
- Inflation layer: Engellerin çevresi, aracın boyutu, güvenlik payı kadar büyütülür. Böylece araç engellere fazla yaklaşmaz.

Global costmap ve local costmap

Global, tüm haritayı kapsar. Local, aracın etrafındaki küçük dinamik bölgeyi kapsar.

6 Robot kol

İleri ve ters kinematik nedir

İleri kinematik

Robot kolunun eklem değerleri biliniyorken uç işlevcinin (end effector) uzaydaki konumunu ve yönelimini hesaplama işlemidir.

Her eklem için bir dönüşüm matrisi vardır.

Bunlar ardışık çarpılarak uç noktanın pozu elde edilir.

Genellikle Denavit-Hartenberg (DH) parametreleri ile modellenir.

Her zaman kesin ve tek sonuç verir, hesaplaması kolaydır.

Ters kinematik

Uç işlevcinin ulaşması istenilen konum ve yönelim verildiğinde eklemlerin değerlerini bulma işlemidir.

Ters kinematik ileriye göre çok daha zordur.

- Denklemler lineer değildir.
- Birden fazla çözüm bulunabilir.
- Hedef çalışma alanının dışındaysa hiç çözüm bulunamayabilir.
- Tekil noktalarda (singularity) sonsuz çözüm olabilir.

Çözüm yöntemleri

Çözüm yöntemleri iki gruba ayrılır:

Analitik (kapalı form): Yöntemler geometrik veya cebirsel olarak doğrudan formül çıkarır. Hızlıdır ama sadece basit yapılarda kullanılabilir.

Sayısal (iteratif): Yöntemler ise Jacobian tabanlıdır. Jacobian tersi, pseudo inverse, damped least squares gibi genel amaçlıdır ama yavaştır.

7 Görüntü işleme

Robotikte görüntü işleme

Robotun kamera yardımıyla aldığı verileri işleyerek anlamlı bilgiler edinir ve bununla çevresini anlamlandırır. Robota görme yeteneği sağlar.

Kullanım alanı

- Nesne tespiti ve tanıma: Ortamdaki nesnelerin ne olduğu, nerede olduğunu belirleme.
- Konum ve yönelim tahmini (pose estimation): Nesnenin 3B konumu ve duruşunu hesaplama, robot kolunun bir nesneyi doğru açıyla kavrayabilmesi.

- Navigasyon ve haritalama: Visual SLAM ve görsel odometri ile robotun hem harita çıkarabilmesi hem de bu haritada konumunu bulması.
- Engel algılama: Stereo kamera veya derinlik kamera ile mesafe bilgisi çıkarılır.
- Şerit ve yol takibi: Otonom araçların şerit çizgilerini tespit edilmesi.

8 Haberleşme

900 MHz 2.4 GHz ve 5 GHz frekans bantları arasındaki temel farklar

Temel fark şudur: Frekans düştükçe menzil ve engel geçirgenliği artar ama veri hızı, bant genişliği azalır.

Sebebi, düşük frekanslı dalgaların serbest uzayda ve engellerin içinden geçerken daha az zayıflamasıdır.

Yüksek frekanslarda ise daha geniş kanallar kullanılabildiği için daha fazla veri taşınabilir.

Özellik	900 MHz	2.4 GHz	5 GHz
Menzil	En uzun (km'ler)	Orta	En kısa
Engel geçirgenliği	En iyi	Orta	Zayıf
Veri hızı	Düşük	Orta	Yüksek
Parazit / kalabalık	Az	Çok yoğun	Az
Anten boyutu	Büyük	Orta	Küçük

900 MHz

Uzun menzil, düşük veri hızı gerektiren alanlarda tercih edilir.

Drone, İHA telemetrisi.

2.4 GHz

Dünya genelinde lisanssız kullanılabilen ISM bandıdır.

Wi-Fi, Bluetooth, Zigbee, RC kumandalar bu bant ile çalışır.

5 GHz

5.8 GHz dâhil.

Yüksek veri hızı gerektiren alanlarda tercih edilir.

Örnek olarak FPV görüntü aktarımı, yüksek hızlı Wi-Fi.

Robotik ve İHA uygulamalarında bu yüzden genelde bantlar birlikte kullanılır.

Örneğin kontrol ve telemetri 900 MHz veya 2.4 GHz üzerinden, video ise 5.8 GHz üzerinden aktarılır.

Line of Sight kavramı nedir

Line of Sight (LoS): Görüş hattı.

Verici ve alıcı arasında hiç engel olmayan doğrudan düz bir sinyal yolunun olması demektir.

Sinyal, yansımaya ve kırınımına uğramadan doğrudan alıcıya ulaşır.

Fresnel bölgesi

Antenlerin birbirini görmesi tek başına yeterli değildir. İki anten arasında elipsoid şeklindeki alanın birinci Fresnel bölgesinin en az %60 engelsiz olması gerekmektedir.

Aksi hâlde sinyali zayıflatır.

Dünya eğriliği

Uzun mesafelerde bu eğrilik görüş hattını keser.

Bu yüzden menzil artırıldığında antenler yükseğe yerleştirilir.

NLoS

Non Line of Sight.

Arada engel olması durumudur. Sinyal, yansıma, kırınım ve saçılma ile ulaşır.

Bu, sinyal zayıflamasına ve çok yollu yayılım (multipath) kaynaklı bozulmalara yol açar.

9 Git ve GitHub

Git nedir

Git, Linux çekirdeğinin geliştirilmesi için oluşturulmuş ücretsiz ve açık kaynaklı dağıtık sürüm kontrol sistemidir.

Distributed Version Control System.

Bir projedeki dosyaları ve zaman içindeki her değişimi kaydeder ve yönetir.

Amacı

- Değişikliklerin geçmişini tutmak: Her commit, projenin o anki anlık görüntüsünü kaydeder.
- Geri dönebilmek: Hatalı değişiklikten zararsız, kolayca geri dönülebilir.
- Paralel geliştirme: Branch (dal) yapısı sayesinde ana kodu bozmadan yeni özellikler veya hata düzeltmeleri denenir, sonra ana dala birleştirilir (merge).
- Ekip çalışmasını kolaylaştırır. Birden fazla çalışan kişinin olduğu durumlarda bir projede aynı anda çalışılmasını sağlar.

Branch

Branch, projenin ana geliştirme hattından ayrılan bağımsız bir çalışma koludur.

- Ana kodu korur.
- Paralel çalışma sağlar.
- Düzenli iş akışı sağlar.
- Kodun incelenmesini kolaylaştırır.
- Kolay deneme ve geri dönüş sağlar.

Temel Git komutları

git init

Bulunulan klasörde yeni boş repository oluşturur. Klasörün içine .git adlı gizli bir dizin ekler.

git clone

```
git clone <url>
```

Uzak bir repository'yi tüm dosyalarıyla ve geçmişiyle bilgisayara kopyalar.

git add

```
git add <dosya>
```

Değiştirilen dosyaları bir sonraki commit'e dâhil edilmek üzere hazırlık alanına (staging area) ekler.

git add . tüm değişiklikleri ekler.

git commit

```
git commit -m "mesaj"
```

Staging area'daki değişiklikleri açıklama ile repository'ye ekler.

git push

Yereldeki repository commit'lerini uzak repository'ye gönderir. Örnek olarak GitHub.

git pull

Uzak repository'deki güncel değişiklikleri indirir.

Teknik olarak git fetch ve git merge işlemidir.

Remote repository

Remote repository (uzak depo), projenin internette veya bir ağ sunucusunda barındırılan kopyasıdır.

Yerel repository geliştiricisinin kendi bilgisayarındayken remote repository tüm ekibin erişebildiği ortak merkezdir.

Varsayılan olarak orijin olarak adlandırılır. Yerel ve uzak repository arasındaki senkronizasyon push, pull, fetch komutlarıyla yapılır.

GitHub, GitLab, Bitbucket gibi platformlar bu remote repository'leri barındırır ve şu rolleri üstlenir:

- Ortak merkez: Ekip üyeleri değişikliklerini buraya gönderir ve başkalarınınkini buradan alır.
- Yedekleme: Proje bulutta saklandığı için yerel bilgisayar arızalanırsa bile kod ve geçmişi saklanır.
- Açık kaynak ve paylaşım: Projeler herkese açık yayınlanabilir, başkaları fork alarak projeye katkıda bulunabilir.
- Otomasyon: GitHub Actions gibi CI/CD araçlarıyla her push sonrasında testler ve derleme otomatik çalıştırılabilir.